

Presentation Guide

Restaurant Table Reservation & Waitlist System

Java Programming - Case Study 70 | Aditya Sunil Chouksey (150096725070)

What this guide gives you

How to structure a 10-minute presentation of this project, exactly what to put on each slide and what to say, how to fold the live demo in, and how to deliver it calmly.

It works together with two other documents: the **Live Demo Script** (every click of the demo) and **Viva Questions & Answers** (what you will be asked afterwards).

1. Choose your format

Ask your faculty which they expect. If they do not say, use Format B - it shows understanding (slides) and proof (demo).

	Format A - demo-led	Format B - slides + demo (recommended)
Shape	One opening line, 10 minutes of live demo, one closing line	4 min of slides, 4.5 min of live demo, 1.5 min of slides
Best when	The panel said "show us the working project"	The panel expects a presentation, or you are not sure
You need	The app only	The app + 9 slides (content in section 2)
Script	Live Demo Script, as written	This guide, sections 2-3
Risk	If the app fails, there is little to fall back on	If the app fails, the slides and screenshots carry you

Format B timeline

Time	Part	Content
0:00 - 4:00	Slides 1-6	Title, problem, objectives and modules, architecture, data structures, key algorithms
4:00 - 8:30	Live demo	Booking + validation, walk-ins, table release -> notification, check-in, reports and restart
8:30 - 10:00	Slides 7-9	Testing and results, conclusion and future scope, thank you / questions

2. Slide-by-slide plan (Format B)

Slide design rules

At most 5 short lines per slide, at least 24 pt text, one idea per slide.

Show diagrams and screenshots, not code. Never read the slide aloud - the slide is for the audience, the "Say" box is for you.

Use one simple theme throughout; put the slide number in a corner so examiners can refer to it ("on slide 5...").

All visuals below are already in the project: docs/uml/ and docs/screenshots/.

Slide 1 - Title

☰ 0:00 - 0:20

On the slide	Visual
Restaurant Table Reservation & Waitlist System Case Study 70 - Java Programming Aditya Sunil Chouksey 150096725070 B.Tech CSE (2025-29), Semester III	ITM Skills University logo (docs/_build/itm-logo.png)
Say "Good morning. I'm Aditya Sunil Chouksey. My case study is a Restaurant Table Reservation and Waitlist System, built in Java with a Swing interface."	

Slide 2 - The problem

☰ 0:20 - 1:00

On the slide	Visual
Bookings + walk-ins + tables that keep changing Paper leads to double bookings Big tables wasted on small groups Walk-ins forgotten or seated out of turn Goal: seat by the rules, tell the host who is next	Optional: a simple restaurant / table icon. Keep it light.
Say "A restaurant host juggles three things: guests who booked, guests who just walk in, and tables that are free, held or occupied. Done on paper, tables get double-booked, a couple ends up at a six-seat table, and walk-ins get skipped. My system applies the seating rules automatically and tells the host exactly who should get a table the moment it frees up."	

Slide 3 - Objectives and modules

☰ 1:00 - 1:40

On the slide	Visual
6 objectives: tables & bookings, waitlist, assign by size, notify, search & sort, GUI 7 modules: Table Setup - Reservation Booking - Waitlist Table Assignment - Availability Notification Reservation Search - Seating Reports	Two short columns, or seven small coloured boxes, one per module.
Say "The brief gave six objectives and seven modules. Every one is implemented - each module is a tab or a service in the application, and I will show most of them live in a minute."	

Slide 4 - Architecture

1:40 - 2:30

On the slide	Visual
GUI -> Service -> Model layers RestaurantManager makes every decision 2 background threads: notifier (2 s) and monitor (every 15 s) Autosave to data/restaurant.dat	docs/uml/architecture.png (full width)

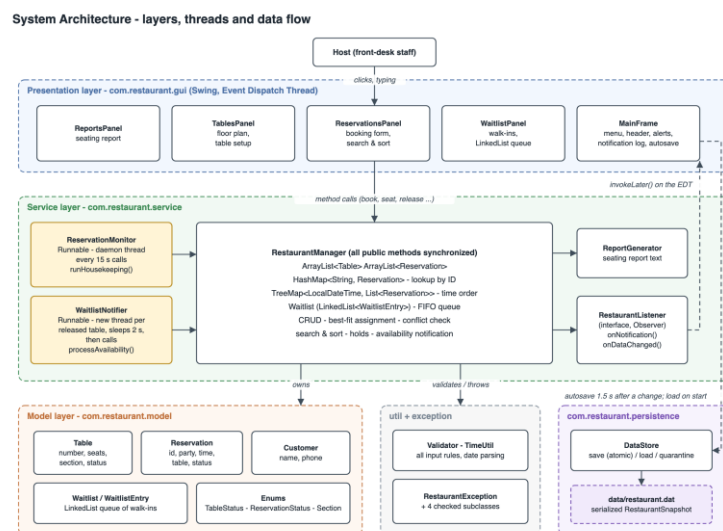


Figure 1: Visual for slide 4

Say

"The code is in layers. The Swing GUI only collects input and shows results; every rule lives in one service class, RestaurantManager. Two background threads - a notifier and a monitor - call the same manager, and data is saved automatically to a file. Because the rules are separate from the screen, they are easy to test."

Slide 5 - Data structures

2:30 - 3:20

On the slide	Visual
ArrayList - all tables and reservations HashMap - reservation by ID, O(1) lookup TreeMap - bookings in time order ("who is due in 30 min?") LinkedList - the walk-in waitlist queue Enum - TableStatus: AVAILABLE, RESERVED, OCCUPIED	A 4-row table: structure holds why. (Copy it from report Table 3.)
Say "I chose each collection for one question. The ArrayList keeps everything. The HashMap finds a booking by its ID instantly. The TreeMap keeps bookings sorted by time, so I can ask who is due in the next half hour. And the waitlist is a LinkedList, because guests join at the end and are served first-come-first-served - exactly what the brief suggests."	

Slide 6 - Key algorithms

3:20 - 4:00

On the slide	Visual
Best fit: smallest table that fits the party No double booking: each party blocks its table 90 min Fair queue: first waiting party that fits gets the table Notification: release -> thread waits 2 s -> hold -> alert	docs/uml/flowchart-waitlist-notification.png (right half: "B. Guests leave")

Flowchart 2 - Walk-in arrival and table-release notification (Modules 3, 4, 5)

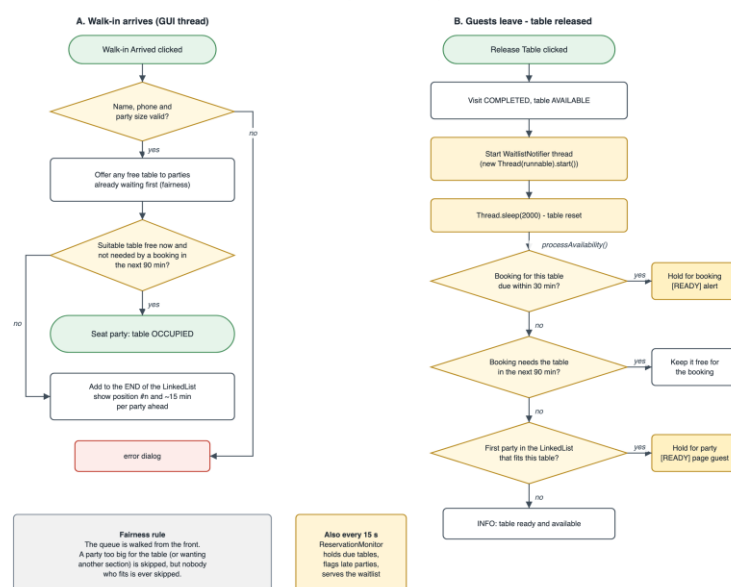


Figure 2: Visual for slide 6

Say

"Three rules drive the system. Best fit gives a party the smallest table that fits. Every party blocks its table for 90 minutes, so two bookings can never overlap. And when guests leave, a background thread waits 2 seconds while the table is reset, then gives it to the booking due next or to the first waiting party that fits - and alerts the host. Let me show you."

Slides 7-9 come after the demo (section 3)

Switch to the application now. Say: "Let me show you the system running."

Slide 7 - Testing and results

8:30 - 9:00

On the slide

16 test cases - all passed

Every module and every validation rule

Errors explain the fix ("15-minute slots", "cannot exceed 20")

Background notification verified at ~2 s

Visual

docs/screenshots/06-validation-party-size.png

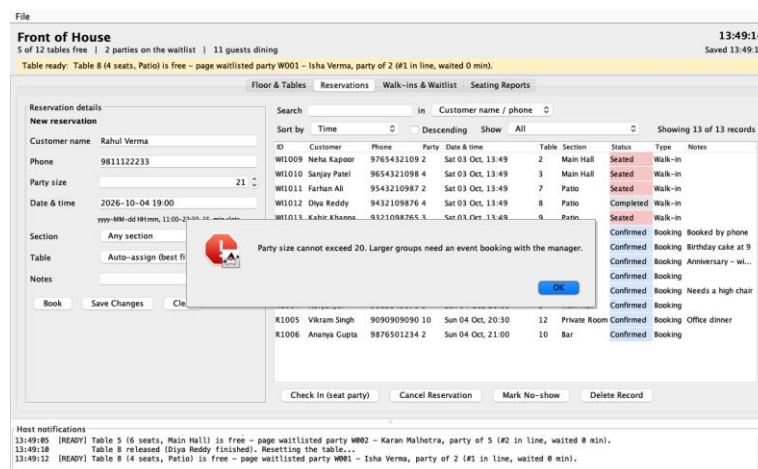


Figure 3: Visual for slide 7

Say

"I tested the finished system against sixteen cases covering every module and every validation rule - including the background notification and restoring data after a restart. All sixteen passed. Bad input never crashes the program; it gets a message that explains the rule."

Slide 8 - Conclusion and future scope

9:00 - 9:40

On the slide	Visual
All 6 objectives, 7 modules, 14 required Java concepts Realistic rules: best fit, fair queue, held tables, autosave Next: database for several front desks, SMS paging, joining tables	None, or a small row of ticks next to the three numbers.
Say "To conclude: the system meets all six objectives, all seven modules and all fourteen Java concepts in the brief. With more time I would move the data to a database so several front desks can share it, page waiting guests by SMS, and allow tables to be joined for large groups."	

Slide 9 - Thank you - questions

☰ 9:40 - 10:00

On the slide	Visual
Thank you Questions? Aditya Sunil Chouksey - 150096725070	Keep the app open behind the slide so you can switch to it for answers.
Say "Thank you. I'm happy to take questions - and I can show anything in the application or the code."	

3. The 4.5-minute live demo (Format B)

This is a shortened version of the **Live Demo Script**. Do its "Before you start" checklist first (fresh sample data, run between 11:00 and 22:30).

Time	Do	Say (short)
4:00 - 4:30	Floor & Tables tab - point at the colours, table 5 (held for W002) and table 1 (held for Meera Joshi).	"Here is the live floor: green free, amber held, red occupied."
4:30 - 5:45	Reservations: Rahul Verma, 9811122233, party 21, tomorrow 19:00 > Book (refused). Party 4, time 19:07 > Book (refused). Time back to 19:00 > Book (confirmed).	"Twenty-one guests: refused. 19:07: we book in 15-minute slots. A party of four at 19:00: best fit chose a four-seat table."
5:45 - 6:30	Walk-ins & Waitlist: Kiran Das, 9877700011, party 2, Any > Walk-in Arrived. Then Zoya Mirza, 9877700022, party 2, Patio > Walk-in Arrived.	"Kiran is seated at once. The patio is full, so Zoya joins the end of the LinkedList queue."
6:30 - 7:30	Floor & Tables: table 8 (Diya Reddy) > Release Table. Wait for the beep and the yellow banner. Then Seat Held Party.	"A thread waits 2 seconds while the table is reset - the screen stays usable - then holds it for Isha, first in line for the patio."
7:30 - 8:00	Reservations: Meera Joshi > Check In.	"Her table was held 30 minutes before her booking. She arrives - checked in."
8:00 - 8:30	Seating Reports > Refresh. Close the window and run ./run.sh again.	"The report summarises the evening. Everything is saved - I restart and it's all back."

Back to slide 7. Say: "So that is the system working. Here is how I tested it."

4. Delivering it well

Voice and pace

- Aim for about 130 words a minute - slower than normal conversation. The "Say" boxes are deliberately shorter than ten minutes of speech, leaving room for clicks, pauses and the demo.
- Pause for a second after each key point ("...so two bookings can never overlap." pause). Pauses sound confident; "um" does not.
- Learn the first 30 seconds word for word. Once you have started well, the rest follows.

Body and screen

- Stand to the side of the screen and face the panel, not the slide. Glance at the laptop, talk to people.
- During the demo, say what you are doing before you click: "Now I release table 8..." Silence while clicking feels long.
- Use the mouse pointer to point; move it slowly and keep it still while you talk about something.
- If you make a typo, fix it calmly - it shows you know the system.

Words that work

Instead of...	Say...
"Basically it just stores the data"	"Reservations are stored in an ArrayList and indexed by ID in a HashMap"
"It sort of picks a table"	"Best fit picks the smallest table that fits the party"
"There is some thread thing"	"A background thread waits 2 seconds, then offers the table"
"I think it works"	"All 16 test cases passed"

5. Rehearsal plan

When	Do this	Done
3 days before	Read Concepts & Study Guide chapters 6 (Collections), 12 (Threads) and 16 (Algorithms). Run the test checklist in the Setup Guide once.	☐
2 days before	Make the 9 slides (if Format B). Do two full timed run-throughs alone. Record one on your phone and watch it back.	☐
1 day before	One run-through in front of a friend who then asks 10 questions from the Viva Q&A. Fix anything you stumbled on.	☐
1 day before	Prepare backups: report PDF/Word and screenshots on the laptop and a USB stick.	☐

When	Do this	Done
On the day	Work through the day-of checklist (section 6) 15 minutes before your slot.	☐

Timing check

If a run-through takes more than 11 minutes, cut the code tour first, then the reports step - never cut the table-release notification, it is the heart of the brief.

6. Day-of checklist

	Item
☐	Laptop charged, charger packed, HDMI / USB-C adapter packed.
☐	Do Not Disturb on; close chat apps and browser tabs; hide desktop clutter.
☐	Connect to the projector early; choose "mirror" display; check the app window fits (it needs at least 1100 x 700).
☐	Inside service hours (11:00-22:30)? Today's sample bookings only appear then.
☐	Start the app (JAVA_HOME=/opt/homebrew/opt/openjdk@21 ./run.sh), then File > Load Sample Data > Yes.
☐	Slides open on slide 1 (Format B); app open on the Floor & Tables tab behind it.
☐	Terminal open in the project folder with ./run.sh ready for the restart step.
☐	Code editor open on RestaurantManager.java, Waitlist.java and WaitlistNotifier.java (for questions).
☐	Sound on - the [READY] alert beeps.
☐	Backup ready: Project_Report.docx at section 5.3 (snapshots).
☐	Water nearby. Breathe out slowly before you start.

7. Handling questions

1. **Listen to the whole question.** Do not start answering halfway.
2. **Repeat it briefly.** "So you are asking why a LinkedList and not an ArrayList?" - it buys thinking time and checks you understood.
3. **Answer in two sentences.** Then stop. If they want more, they will ask.
4. **Show it.** "Let me show you" - switch to the app or the code. This is the most convincing answer there is.
5. **If you do not know:** say what you do know and how you would find out. Never guess or bluff.

The ten most likely questions

Question	Your answer in one line
Why a LinkedList for the waitlist?	Guests join at the end and leave from the front or middle - $O(1)$ with a LinkedList; an ArrayList would shift elements.
Why a TreeMap?	It keeps bookings sorted by time, so <code>subMap()</code> gives "who is due in the next 30 minutes" directly.
Why a HashMap?	Finding a reservation by ID is $O(1)$ instead of scanning every booking.
How do you prevent double booking?	Each party blocks its table for 90 minutes; a new booking conflicts if the time intervals overlap.
How does the notification work?	Releasing a table starts a thread that sleeps 2 s, then holds the table for the next booking or the first waiting party that fits, and alerts the host.
Why synchronized?	The GUI and two threads change the same data; synchronized lets only one in at a time, so there are no race conditions.
How do threads update the GUI?	Through the <code>RestaurantListener</code> interface; <code>MainFrame</code> uses <code>SwingUtilities.invokeLater</code> so updates run on the Event Dispatch Thread.
What happens with bad input?	<code>Validator</code> throws a checked <code>InvalidReservationException</code> ; the GUI catches it and shows the message. Nothing is saved.
How is data saved?	Serialization to a temporary file, then an atomic move over <code>data/restaurant.dat</code> , 1.5 s after each change.
What would you improve?	A database for several front desks, SMS paging for waiting guests, joining tables for large groups.

More questions with fuller answers: *Viva Questions & Answers*.

8. Common mistakes to avoid

- Reading the slides aloud, or showing slides full of code.
- Starting the demo without resetting the data - leftover rehearsal bookings make the steps behave differently.
- Clicking in silence. Narrate every action.

- Running over time. Practise with a timer; know what to cut (section 5).
- Saying "it just works" - always name the structure or the rule: HashMap, best fit, synchronized.
- Panicking if something fails. Say "let me show you the screenshot instead" and carry on.

9. One-page cue card (print this)

Time	Part	Key words
0:00	S1 Title	name - Restaurant Reservation & Waitlist - Java Swing
0:20	S2 Problem	bookings + walk-ins + changing tables - double booking - fairness
1:00	S3 Objectives	6 objectives - 7 modules - all implemented
1:40	S4 Architecture	GUI -> Service -> Model - RestaurantManager - 2 threads - autosave
2:30	S5 Data structures	ArrayList all - HashMap by ID - TreeMap by time - LinkedList queue
3:20	S6 Algorithms	best fit - 90-min window - fair queue - thread 2 s
4:00	DEMO floor	colours - table 5 W002 - table 1 Meera
4:30	DEMO booking	Rahul 21 refused - 19:07 refused - 4 at 19:00 -> 4-seater
5:45	DEMO walk-ins	Kiran seated - Zoya patio -> queue
6:30	DEMO release	table 8 -> 2 s -> Isha -> Seat Held Party
7:30	DEMO check-in	Meera -> Check In
8:00	DEMO reports	Refresh - close - ./run.sh - restored
8:30	S7 Testing	16 cases - all passed - clear errors
9:00	S8 Conclusion	6 / 7 / 14 - future: database, SMS, join tables
9:40	S9 Questions	thank you - app open behind

Numbers to remember

90-minute dining window | 30-minute hold | 15 minutes late = warning | 2-second reset
| 15-second sweep | party 1-20 | 11:00-22:30, 15-minute slots | 16 tests passed